Architecture design patterns that support operational excellence

When you design workload architectures, you should use industry patterns that address common challenges. Patterns help you make intentional tradeoffs and optimize for desired outcomes. They also help you mitigate risks that can impact reliability, security, performance, and cost. Because operations span all those areas, unmanaged risks eventually surface as operational toil or incidents. These patterns are proven in real-world cloud environments, scale with modern operating models, and are inherently vendor agnostic. Standardizing on well-known patterns is itself a practice of operational excellence.

Many patterns reinforce one or more Azure Well-Architected pillars. For operational excellence specifically, patterns often provide topologies that enable safe deployment practices, incremental evolution, controlled migration, and observability.

The following table summarizes architecture design patterns that support operational excellence goals.

Expand table

Pattern	Summary
Anti-Corruption Layer	Protects new system components from the behavior or implementation choices of legacy systems by adding a mediator layer to proxy interactions between legacy and new components. This pattern helps ensure that new component design remains uninfluenced by legacy implementations that might have different data models or business rules when you integrate with these legacy systems. The pattern is especially useful in gradual system migrations. It reduces technical debt in new components while still supporting existing components.
Choreography	Coordinates the behavior of autonomous distributed components in a workload by using decentralized, event-driven communication. This pattern can be useful when you expect to update or replace services frequently during the lifecycle of a workload. Because the distributed components are autonomous, you can modify the workload with less overall change to the system.
Compute Resource Consolidation	Optimizes and consolidates compute resources by increasing density. This pattern combines multiple applications or components of a workload on a shared infrastructure. The consolidation leads to a more homogenous compute platform, which can simplify management and observability, reduce disparate approaches to operational tasks, and reduce the amount of tooling that's required.
Deployment Stamps	Provides an approach for releasing a specific version of an application and its infrastructure as a controlled unit of deployment, based on the assumption that the same or different versions will be deployed concurrently. This pattern aligns

Pattern	Summary
	with immutable infrastructure goals, supports advanced deployment models, and can facilitate safe deployment practices.
External Configuration Store	Extracts configuration to a service that's externalized to the application to support dynamic updates to configuration values without requiring code changes or application redeployment. This separation of application configuration from application code supports environment-specific configuration and applies versioning to configuration values. External configuration stores are also a common place to manage feature flags to enable safe deployment practices.
Gateway Aggregation	Simplifies client interactions with your workload by aggregating calls to multiple backend services in a single request. This topology enables backend logic to evolve independently from clients, allowing you to change the chained service implementations, or even data sources, without needing to change client touchpoints.
Gateway Offloading	Offloads request processing to a gateway device before and after forwarding the request to a backend node. Adding an offloading gateway to the request process enables you to manage the configuration and upkeep of the offloaded functionality from single point instead of managing it from multiple nodes.
Gateway Routing	Routes incoming network requests to various backend systems based on request intents, business logic, and backend availability. Gateway routing enables you to decouple requests from backends, which in turn enables your backends to support advanced deployment models, platform transitions, and a single point of management for domain name resolution and encryption in transit.
Health Endpoint Monitoring	Provides a way to monitor the health or status of a system by exposing an endpoint that's specifically designed for that purpose. Standardizing which health endpoints to expose, and the level of analysis in the results, across your workload can help you triage issues.
Messaging Bridge	Provides an intermediary to enable communication between messaging systems that are otherwise incompatible because of protocol or format. This decoupling provides flexibility when you transition messaging and eventing technology within your workload or when you have heterogeneous requirements from external dependencies.
Publisher/Subscriber	Decouples components of an architecture by replacing direct client-to-service or client-to-services communication with communication via an intermediate message broker or event bus. This layer of indirection can enable you to safely change the implementation on either the publisher or subscriber side without needing to coordinate changes to both components.
Quarantine	Ensures external assets meet a team-agreed quality level before being authorized to consume them in the workload. Automation and consistency on these checks are a part of the workload's software development lifecycle and safe deployment practices (SDP).

Pattern	Summary
Sidecar	Extends the functionality of an application by encapsulating nonprimary or cross-cutting tasks in a companion process that exists alongside the main application. This pattern provides an approach to implementing flexibility in tool integration that might enhance the application's observability without requiring the application to take direct implementation dependencies. It enables the sidecar functionality to evolve independently and be maintained independently of the application's lifecycle.
Strangler Fig	Provides an approach for systematically replacing the components of a running system with new components, often during a migration or modernization of the system. This pattern provides a continuous improvement approach, in which incremental replacement with small changes over time is preferred rather than large systemic changes that are riskier to implement. This pattern also supports safe decommissioning: legacy endpoints can be measured, drained, and removed only after replacement flows meet reliability and observability targets.

Next steps

Review the Architecture design patterns that support the other Azure Well-Architected Framework pillars:

- Architecture design patterns that support reliability
- Architecture design patterns that support security
- Architecture design patterns that support cost optimization
- Architecture design patterns that support performance efficiency

Last updated on 11/08/2025